

A Technical Introduction to Sparse attention transformer

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Section 1

Alternative attention graphs The standard attention graph is either all-to-all or causal, both of which scales as $O(N^2)$ where N is the number of tokens in a sequence. Reformer (2020) reduces the computational load from $O(N^2)$ to $O(N \ln N)$ by using locality-sensitive hashing and reversible layers. Sparse attention uses attention graphs that grows slower than $O(N^2)$. For example, BigBird (2020) uses random small-world networks which grows as $O(N)$. Ordinary transformers require a memory size that is quadratic in the size of the context window. Attention-free transformers reduce this to a linear dependence while still retaining the advantages of a transformer by linking the key to the value.

Section 2

AI boom era As early as spring 2017, even before the "Attention is all you need" preprint was published, one of the co-authors applied the "decoder-only" variation of the architecture to generate fictitious Wikipedia articles. Transformer architecture is now used alongside many generative models that contribute to the ongoing AI boom. The "reference implementation" of the original Transformer was written in a TensorFlow library. In language modelling, ELMo (2018) was a bi-directional LSTM that produces contextualized word embeddings, improving upon the line of research from bag of words and word2vec. It was followed by BERT (2018), an encoder-only transformer model. In October 2019, Google started using BERT to process search queries. In 2020, Google Translate replaced the previous RNN-encoder-RNN-decoder model by a transformer-encoder-RNN-decoder model. Starting in 2018, the OpenAI GPT series of decoder-only transformers became state of the art in natural language generation. At the end of 2022, ChatGPT, a chatbot based on a fine-tuned variant of GPT-3.5, became unexpectedly popular, triggering a boom around large language models. Transformers have been applied in modalities beyond text. Four days after the publication of "Attention is All You Need", a multimodal transformer architecture, MultiModel, was published by most authors of that paper. Other examples include the vision transformer, speech recognition, robotics, and multimodal. The vision transformer, in turn, stimulated new developments in

convolutional neural networks. Image and video generators like DALL-E (2021), Stable Diffusion 3 (2024), and Sora (2024), use transformers to analyse input data (like text prompts) by breaking it down into "tokens" and then calculating the relevance between each token using self-attention, which helps the model understand the context and relationships within the data.

Section 3

Predecessors For many years, sequence modelling and generation was done by using plain recurrent neural networks (RNNs). A well-cited early example was the Elman network (1990). In theory, the information from one token can propagate arbitrarily far down the sequence, but in practice the vanishing-gradient problem leaves the model's state at the end of a long sentence without precise, extractable information about preceding tokens. A key breakthrough was LSTM (originally described in a 1995 technical report and formally published in 1997), an RNN that introduced gating mechanisms to mitigate the vanishing gradient problem, allowing efficient learning of long-sequence modelling. One key architectural element was the use of multiplicative gating units, in which the outputs of some neurons modulate the outputs of others. These multiplicative units are conceptually distinct from the additive attention mechanism later introduced for sequence-to-sequence models. Neural networks using multiplicative units were later called sigma-pi networks or higher-order networks. LSTM became the standard architecture for long sequence modelling until the 2017 publication of transformers. However, LSTM still used sequential processing, like most other RNNs. Specifically, RNNs operate one token at a time from first to last; they cannot operate in parallel over all tokens in a sequence. Modern transformers overcome this problem, but unlike RNNs, they require computation time that is quadratic in the size of the context window. The linearly scaling fast weight controller (1992) learns to compute a weight matrix for further processing depending on the input. One of its two networks has "fast weights" or "dynamic links" (1981). A slow neural network learns by gradient descent to generate keys and values for computing the weight changes of the fast neural network which computes answers to queries. This was later shown to be equivalent to the unnormalized linear transformer.

Section 4

```
/* second sublayer */ z_d_copy ← copy(z_d) for each t in
1:length(z_d) do z_d[t] ← layer.layer_norm(z_d[t]) z_d ←
layer.multihead_attention(z_d, z_e, z_e) for each t in
1:length(z_d) do z_d[t] ← z_d[t] + z_d_copy[t]
```

Section 5

```
/* third sublayer */ z_d_copy ← copy(z_d) for each t in
1:length(z_d) do z_d[t] ← layer.layer_norm(z_d[t]) z_d ←
```

```
layer.feedforward(z_d) for each t in 1:length(z_d) do z_d[t]
← z_d[t] + z_d_copy[t]
```

Section 6

Tokenization As the transformer architecture natively consists of operations over numbers (matrix multiplications, dot products, activation functions) rather than over text, there must first be a mapping from any input text to some numerical representation. This happens in three steps. First, the input text is treated by a preprocessor, which performs both textual transformations and splits the text into coarse-grained segments called pretokens. The latter is referred to as pretokenization. Second, each pretoken is segmented further into tokens by a tokenizer that expects to only see pretokens output by its preprocessor. Each token it produces is a string of one or more characters belonging to a finite set of strings called the vocabulary V . Third, because the vocabulary is finite and known beforehand, each token can be assigned an integer identifier, and this mapping is applied to the sequence of tokens to represent any input text as a numerical sequence. Since this mapping is bijective, the output side can produce a sequence of integer identifiers which can then be turned back into tokens. After undoing some of the preprocessing, the result is again legible text. Training a tokenizer (sometimes referred to as vocabularization) means finding a suitable vocabulary V , but also learning how to use it, since any given string s of length $|s|$ has $2^{|s|-1}$ hypothetical segmentations, some of which containing segments that are not in the vocabulary. The most important hyperparameter during vocabularization is the vocabulary size $|V|$: when it is small, the learned vocabulary generally consists of characters and smaller strings, and words will be segmented into many tokens. At larger sizes, it becomes affordable to dedicate tokens to full words, although depending on the preprocessor and tokenizer, it is not necessarily the case that large vocabularies will always use the largest token(s) available to segment a word. Because tokens are not always full words, they may also be referred to as subwords and tokenization algorithms may be referred to as subword tokenizers. This is also to differentiate these systems from traditional terminology used in older information retrieval and natural language processing systems, where "tokenization" was used to denote what is today called "pretokenization" (very crudely: splitting into words). In tokenizers that produce tokens that are not part of the vocabulary, a special token that does belong to the vocabulary is used as a generic stand-in, written as "[UNK]" for "unknown". In principle, any string could be hidden by such an [UNK]. Indeed, in information retrieval, pretokenizers were themselves used as tokenizers (and also called "tokenizers") with a word-level vocabulary that contained an [UNK]. Commonly used subword tokenization

algorithms are byte pair encoding (BPE) and the unigram language model (ULM), which each include a vocabularization algorithm and a dedicated segmentation algorithm. There also exist several segmentation algorithms that require no learning and can be applied given a vocabulary (produced by BPE or ULM, for example), like greedily recognising tokens in a pretoken by moving through it left-to-right. Well-known software implementations of subword tokenizers are Hugging Face's tokenizers Python package implemented in Rust, and the sentencepiece Python package implemented in C++. The latter package is named as such because one of its configuration options allows disabling the built-in pretokenizer, hence effectively making entire sentences a pretoken and thus having the tokenizer see entire sentences, rather than individual words.